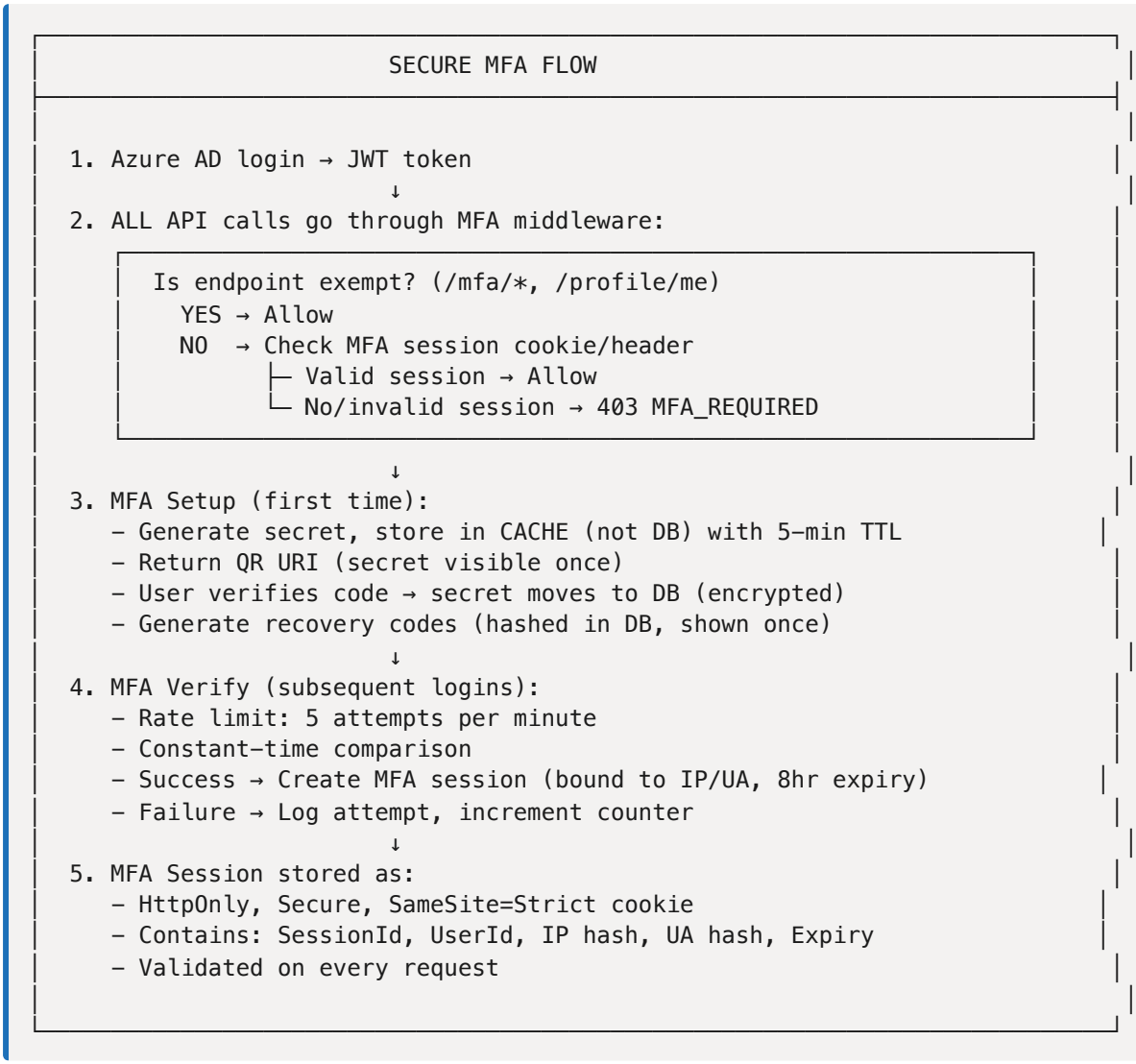
MFA Implementation with Microsoft Authenticator (TOTP)

Document Version: 1.0 | **Date:** February 2026

Overview

This document outlines the implementation plan for adding Multi-Factor Authentication (MFA) to the Gambling Levy Platform using Time-based One-Time Passwords (TOTP), compatible with Microsoft Authenticator and other authenticator apps.

Architecture



Security Considerations

Identified Risks and Mitigations

Risk	Mitigation	Priority
API bypass (MFA only enforced on frontend)	Backend MFA middleware on all endpoints	**Critical**
Secret exposure	Encrypt at rest, show once, HTTPS only	**Critical**
Session theft	Bind to IP/UA, short expiry, HttpOnly cookie	**High**
Brute force	Rate limiting (5/min), lockout after failures	**High**
Setup race condition	Cache-based pending setup, not DB	**Medium**
Recovery code weakness	Long codes, bcrypt hash, aggressive rate limit	**Medium**
Sensitive action abuse	Require re-verification to disable MFA	**Medium**

Implementation Phases

Phase 1: Backend Domain Layer

1.1 Create Domain Entities

Task	File	Description
Create `UserMfaId`	`Domain/Users/UserMfaId.cs`	Strongly-typed ID
Create `UserMfa` entity	`Domain/Users/UserMfa.cs`	MFA config per user (secret, enabled, recovery codes)
Create `MfaSessionId`	`Domain/Users/MfaSessionId.cs`	Strongly-typed ID
Create `MfaSession` entity	`Domain/Users/MfaSession.cs`	Verified MFA sessions (bound to IP/UA)

1.2 Create Domain Exceptions

Task	File
Create `MfaException` base	`Application/Mfa/Exceptions/MfaException.cs`
Create `MfaRequiredException`	`Application/Mfa/Exceptions/MfaRequiredException.cs`
Create `InvalidMfaCodeException`	`Application/Mfa/Exceptions/InvalidMfaCodeException.cs`
Create `MfaRateLimitException`	`Application/Mfa/Exceptions/MfaRateLimitException.cs`
Create `MfaSetupPendingException`	`Application/Mfa/Exceptions/MfaSetupPendingException.cs`
Create `MfaAlreadyEnabledException`	`Application/Mfa/Exceptions/MfaAlreadyEnabledException.cs`

Phase 2: Backend Infrastructure Layer

2.1 Dependencies

Task	Description
Add `Otp.NET` NuGet package	TOTP generation/validation (RFC 6238)
Add `BCrypt.Net-Next` NuGet package	Recovery code hashing

2.2 Services

Task	File	Desc
Create `ITotpService` interface	`Application/Common/Interfaces/ITotpService.cs`	Interf TOTP opera
Create `TotpService`	`Infrastructure/Mfa/TotpService.cs`	Gene secre valid
Create `IMfaEncryptionService` interface	`Application/Common/Interfaces/IMfaEncryptionService.cs`	Secr encry interf
Create `MfaEncryptionService`	`Infrastructure/Mfa/MfaEncryptionService.cs`	Encr secre Data API
Create `IMfaRateLimitService` interface	`Application/Common/Interfaces/IMfaRateLimitService.cs`	Rate interf
Create `MfaRateLimitService`	`Infrastructure/Mfa/MfaRateLimitService.cs`	Track enfor limit
Create `IMfaSessionService` interface	`Application/Common/Interfaces/IMfaSessionService.cs`	Sess mana interf
Create `MfaSessionService`	`Infrastructure/Mfa/MfaSessionService.cs`	Crea sessi to IP

2.3 Repositories

Task	File
Create `IUserMfaRepository`	`Application/Common/Interfaces/Repositories/IUserMfaRepository`
Create `UserMfaRepository`	`Infrastructure/Persistence/Repositories/UserMfaRepository.cs`
Create `IMfaSessionRepository`	`Application/Common/Interfaces/Repositories/IMfaSessionReposi`
Create `MfaSessionRepository`	`Infrastructure/Persistence/Repositories/MfaSessionRepository`

2.4 Database Configuration

Task	File
Create `UserMfaConfiguration`	`Infrastructure/Persistence/Configurations/UserMfaConfigura`
Create `MfaSessionConfiguration`	`Infrastructure/Persistence/Configurations/MfaSessionConfigu`
Add DbSets to `ApplicationDbContext`	`Infrastructure/Persistence/ApplicationDbContext.cs`
Create migration	Run `dotnet ef migrations add AddMfaTables`

2.5 Service Registration

Task	File	Description
Register MFA services	`Infrastructure/ConfigureServices.cs`	Register all MFA services in DI

Phase 3: Backend Application Layer

3.1 DTOs

Task	File	Description
Create `MfaSetupResponseDto`	`Application/Mfa/Dtos/MfaSetupResponseDto.cs`	QR URI, manual key
Create `MfaStatusDto`	`Application/Mfa/Dtos/MfaStatusDto.cs`	Enabled status, setup date
Create `MfaVerifyRequestDto`	`Application/Mfa/Dtos/MfaVerifyRequestDto.cs`	6-digit code input
Create `MfaVerifyResponseDto`	`Application/Mfa/Dtos/MfaVerifyResponseDto.cs`	Success, session expiry
Create `MfaRecoveryCodesDto`	`Application/Mfa/Dtos/MfaRecoveryCodesDto.cs`	List of recovery codes

3.2 Commands

Task	File
Create `SetupMfaCommand`	`Application/Mfa/Commands/SetupMfaCommand.cs`
Create `VerifyMfaSetupCommand`	`Application/Mfa/Commands/VerifyMfaSetupCommand.cs`
Create `VerifyMfaCommand`	`Application/Mfa/Commands/VerifyMfaCommand.cs`
Create `VerifyRecoveryCodeCommand`	`Application/Mfa/Commands/VerifyRecoveryCodeCommand.`
Create `DisableMfaCommand`	`Application/Mfa/Commands/DisableMfaCommand.cs`
Create `RegenerateRecoveryCodesCommand`	`Application/Mfa/Commands/RegenerateRecoveryCodesComm`

3.3 Queries

Task	File	Description
Create `GetMfaStatusQuery`	`Application/Mfa/Queries/GetMfaStatusQuery.cs`	Check if MFA enabled for user

Phase 4: Backend API Layer

4.1 Controller

Task	File	Description
Create `MfaController`	`Api/Controllers/MfaController.cs`	All MFA endpoints

Endpoints:

Method	Route	Description	Auth
GET	`/mfa/status`	Get MFA status	JWT only
POST	`/mfa/setup`	Initiate MFA setup	JWT only
POST	`/mfa/setup/verify`	Complete MFA setup	JWT only
POST	`/mfa/verify`	Verify TOTP code	JWT only
POST	`/mfa/verify/recovery`	Verify recovery code	JWT only
POST	`/mfa/disable`	Disable MFA	JWT + TOTP code
POST	`/mfa/recovery-codes/regenerate`	Regenerate recovery codes	JWT + TOTP code

4.2 Middleware & Security

Task	File	Description	Secu
Create `MfaRequiredMiddleware`	`Api/Middleware/MfaRequiredMiddleware.cs`	Check MFA session for all protected endpoints	**Prev bypa**
Create `MfaExemptAttribute`	`Api/Attributes/MfaExemptAttribute.cs`	Mark endpoints that don't require MFA	For / /prof
Create `MfaSessionCookieHandler`	`Api/Middleware/MfaSessionCookieHandler.cs`	Set/read MFA session cookie	Http Sam
Update `Program.cs`	`Api/Program.cs`	Register MFA middleware	After Use/

4.3 Exception Handling

Task	File	Description
Update exception handler	`Api/Middleware/ExceptionHandler.cs`	Handle MFA exceptions, return proper error codes

Error Responses:

Exception	HTTP Status	Error Code
`MfaRequiredException`	403	`MFA_REQUIRED`
`InvalidMfaCodeException`	401	`INVALID_MFA_CODE`
`MfaRateLimitException`	429	`MFA_RATE_LIMITED`
`MfaAlreadyEnabledException`	409	`MFA_ALREADY_ENABLED`

Phase 5: Backend Security Hardening

Task	Description	Priority
Constant-time comparison	Use `CryptographicOperations.FixedTimeEquals` for TOTP	**Critical**
Secret encryption	Encrypt TOTP secret at rest using Data Protection API	**Critical**
Rate limiting	5 attempts per minute per user, using IMemoryCache	**High**
Session binding	Hash IP + UserAgent, validate on each request	**High**
Recovery code hashing	BCrypt hash, one-time use, delete after use	**High**
Lockout mechanism	Lock MFA after 5 failed recovery attempts (30 min)	**Medium**
Audit logging	Log MFA events (setup, verify, disable)	**Medium**

Phase 6: Frontend Implementation

6.1 Dependencies

Task	Description
Add `qrcode.react`	QR code generation
Add `react-otp-input`	6-digit code input component

6.2 Context & Hooks

Task	File	Description
Create `MfaProvider`	`components/MfaProvider/index.tsx`	MFA context (status, verify, setup)
Create `useMfa` hook	`components/MfaProvider/index.tsx`	Hook to access MFA context
Create MFA queries	`components/MfaProvider/queries.ts`	API calls for MFA endpoints

6.3 Pages

Task	File	Description
Create MFA Setup page	`views/Mfa/Setup/index.tsx`	QR code, manual key, verify input
Create MFA Verify page	`views/Mfa/Verify/index.tsx`	6-digit input, recovery code option
Create MFA Settings page	`views/Settings/Security/index.tsx`	Enable/disable, view recovery codes
Create Recovery Codes display	`views/Mfa/RecoveryCodes/index.tsx`	Show codes after setup, download option

6.4 Auth Flow Updates

Task	File	Description	Security
Update `AuthProvider`	`components/AuthProvider/index.tsx`	Check MFA status after login, redirect to setup/verify	Enforce MFA
Update axios interceptor	`components/App/index.tsx`	Handle 403 MFA_REQUIRED, redirect to /mfa/verify	**Prevent bypass**
Add MFA routes	`Routes.tsx`	Add /mfa/setup, /mfa/verify routes	Public after JWT auth

6.5 UI Components

Task	File	Description
Create `OtpInput` component	`components/OtpInput/index.tsx`	6-digit input with auto-focus
Create `RecoveryCodesList`	`components/RecoveryCodesList/index.tsx`	Display/download recovery codes

Phase 7: Testing

7.1 Unit Tests

Task	File	Description
Test `TotpService`	`Tests.Unit/Mfa/TotpServiceTests.cs`	Secret generation, validation
Test `MfaEncryptionService`	`Tests.Unit/Mfa/MfaEncryptionServiceTests.cs`	Encrypt/decrypt
Test `MfaRateLimitService`	`Tests.Unit/Mfa/MfaRateLimitServiceTests.cs`	Rate limiting logic

7.2 Integration Tests

Task	File	Description
Test MFA setup flow	`Tests.Integration/Mfa/MfaSetupTests.cs`	Full setup flow
Test MFA verify flow	`Tests.Integration/Mfa/MfaVerifyTests.cs`	Login verification
Test MFA middleware	`Tests.Integration/Mfa/MfaMiddlewareTests.cs`	API protection
Test rate limiting	`Tests.Integration/Mfa/MfaRateLimitTests.cs`	Lockout behavior

Phase 8: Documentation & Deployment

Task	Description
Update API documentation	Document all MFA endpoints
Create user guide	How to set up Microsoft Authenticator
Update deployment config	Ensure Data Protection keys are persisted
Plan rollout	Phased rollout strategy (optional MFA → required MFA)

File Structure

```
api/src/
├── Domain/Users/
│   ├── UserMfa.cs
│   ├── UserMfaId.cs
│   ├── MfaSession.cs
│   └── MfaSessionId.cs
├── Application/
│   ├── Common/Interfaces/
│   │   ├── ITotpService.cs
│   │   ├── IMfaEncryptionService.cs
│   │   ├── IMfaRateLimitService.cs
│   │   ├── IMfaSessionService.cs
│   │   └── Repositories/
│   │       ├── IUserMfaRepository.cs
│   │       └── IMfaSessionRepository.cs
│   └── Mfa/
│       ├── Commands/
│       │   ├── SetupMfaCommand.cs
│       │   ├── VerifyMfaSetupCommand.cs
│       │   ├── VerifyMfaCommand.cs
│       │   ├── VerifyRecoveryCodeCommand.cs
│       │   ├── DisableMfaCommand.cs
│       │   └── RegenerateRecoveryCodesCommand.cs
│       ├── Queries/
│       │   └── GetMfaStatusQuery.cs
│       ├── Dtos/
│       │   ├── MfaSetupResponseDto.cs
│       │   ├── MfaStatusDto.cs
│       │   ├── MfaVerifyRequestDto.cs
│       │   ├── MfaVerifyResponseDto.cs
│       │   └── MfaRecoveryCodesDto.cs
│       └── Exceptions/
│           ├── MfaException.cs
│           ├── MfaRequiredException.cs
│           ├── InvalidMfaCodeException.cs
│           ├── MfaRateLimitException.cs
│           ├── MfaSetupPendingException.cs
│           └── MfaAlreadyEnabledException.cs
├── Infrastructure/
│   ├── Mfa/
│   │   ├── TotpService.cs
│   │   ├── MfaEncryptionService.cs
│   │   ├── MfaRateLimitService.cs
│   │   └── MfaSessionService.cs
│   └── Persistence/
│       ├── Configurations/
│       │   ├── UserMfaConfiguration.cs
│       │   └── MfaSessionConfiguration.cs
│       └── Repositories/
│           ├── UserMfaRepository.cs
│           └── MfaSessionRepository.cs
└── Api/
    ├── Controllers/
    │   └── MfaController.cs
    ├── Middleware/
    │   ├── MfaRequiredMiddleware.cs
```

```
    │   └── MfaSessionCookieHandler.cs
    └── Attributes/
        └── MfaExemptAttribute.cs

client/src/
├── components/
│   ├── MfaProvider/
│   │   ├── index.tsx
│   │   └── queries.ts
│   ├── OtpInput/
│   │   └── index.tsx
│   └── RecoveryCodesList/
│       └── index.tsx
└── views/
    └── Mfa/
        ├── Setup/
        │   └── index.tsx
        ├── Verify/
        │   └── index.tsx
        └── RecoveryCodes/
            └── index.tsx
```

Acceptance Criteria

1. Users can set up MFA using Microsoft Authenticator (or any TOTP app)
2. Users must verify MFA on every login after setup
3. Recovery codes are provided for account recovery
4. MFA is enforced at the API level (cannot be bypassed)
5. Rate limiting prevents brute force attacks
6. MFA sessions are bound to device (IP/UserAgent)
7. Sensitive actions (disable MFA, regenerate codes) require re-verification

---

Estimated Effort

Phase	Effort
Phase 1: Domain Layer	0.5 days
Phase 2: Infrastructure Layer	1.5 days
Phase 3: Application Layer	1 day
Phase 4: API Layer	1 day
Phase 5: Security Hardening	0.5 days
Phase 6: Frontend	2 days
Phase 7: Testing	1.5 days
Phase 8: Documentation	0.5 days
Total	**8.5 days**